Ch 03: `if else` and Nested Decision Structures

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

The `if` Statement (1 of 2)

- **Control structure**: logical design that controls order in which set of statements execute
- **Sequence structure**: set of statements that execute in the order they appear
- **Decision structure**: specific action(s) performed only if a condition exists
  - Also known as selection structure

Concept

The `if` statement is used to create a decision structure, which allows a program to have more than one path of execution. The `if` statement causes one ore more statement to execute only when a Boolean expression is true.

EAST TEXAS A&M

The `if` Statement (2 of 2)

- Actions can be **conditionally executed**
  - Performed only when condition is true
  - In Python, a **block** of actions is identified using indentation
- **Single alternative decision structure** provides only one alternative path of execution
  - If condition is not true, exit the structure
- First line known as the `if` clause
  - Includes the keyword `if` followed by condition
    - The condition can be `True` or `False` (boolean condition)
    - When the `if` statement executes, the condition is tested, if `True` the **block** of statements are executed.

Python Decision Structure

```
if condition:
    statement
    statement
    etc.
```



EAST TEXAS A&M

The `if` Statement Flowchart

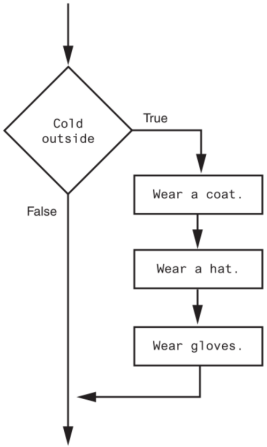


Figure 1: A decision structure that performs three actions if it is cold outside (Gaddis, 2021, pg. 143)

EAST TEXAS A&M

Boolean Expressions and Relational Operators (1 of 3)

- **Boolean expression**: expression tested by `if` statement to determine if it is `True` or `False`
  - Example: `a > b`
    - `True` if `a` is greater than `b`; `false` otherwise
- **Relational operator**: determines whether a specific relationship exists between two values
  - Example: greater than >



EAST TEXAS A&M

Boolean Expressions and Relational Operators (2 of 3)

Table 1: Boolean expressions using relational operators

Expression	Meaning
`x > y`	Is `x` greater than `y`?
`x < y`	Is `x` less than `y`?
`x >= y`	Is `x` greater than or equal to `y`?
`x <= y`	Is `x` less than or equal to `y`?
`x == y`	Is `x` equal to `y`?
`x != y`	Is `x` not equal to `y`?



EAST TEXAS A&M

Boolean Expressions and Relational Operators (3 of 3)

- Any relational operator can be used in a decision block
  - Example: `if balance == 0`
  - Example: `if payment != balance`
- It is possible to have a block of statements inside another block (nested)
  - Example: `if` statement inside a function
  - Statements in inner block must be indented with respect to the outer block

EAST TEXAS A&M

The `if - else` Statement

- **Dual alternative decision structure** two possible paths of execution
  - One taken if the condition is `True`, and the other if the condition is false
  - `if` clause and `else` clause must be aligned
  - Statements must be consistently indented

Python Alternative Decision Structure

```
if condition:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```



The `if - else` Statement Flowchart

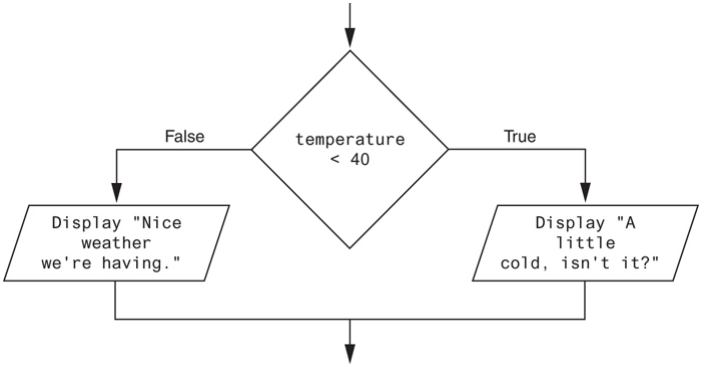


Figure 2: A dual alternative decision structure (Gaddis, 2021, pg.150)

EAST TEXAS A&M

The if - else Statement Execution and Indentation

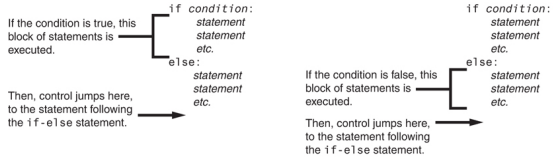


Figure 3: Conditional execution in an if-else statement (Gaddis, 2021, pg.151)

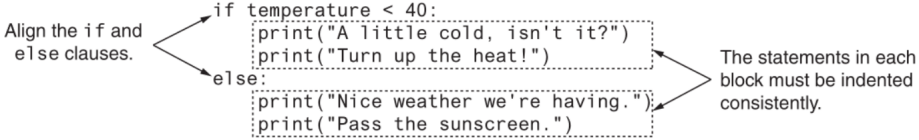


Figure 4: Indentation with an if-else statement (Gaddis, 2021, pg.151)

EAST TEXAS A&M

Comparing Strings

- Strings can be compared using the == and the != boolean operators
- String comparisons are case sensitive

- Strings can be compared using >, <, >=, <=
  - Compared character by character based on ASCII values for each character (e.g. alphabetical ordering)
  - If shorter word is a substring of longer word, longer word is greater than shorter word

Python String Comparisons

Python allows you to compare strings. This allows you to create decision structures that test the value of a string.

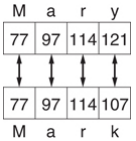


Figure 5: Comparing each character in a string (Gaddis, 2021, pg.155)

EAST TEXAS A&M

Nested Decision Structures and the `if-elif-else` Statement (1 of 2)

- Commonly needed in programs
- Example:
  - Determine if someone qualifies for a loan, they must meet two conditions:
    - Must earn at least $30,000 / year
    - Must have been employed for at least two years
  - Check first condition, and if its true, check second condition

Nested Decision Structures

To test more than one condition, a decision structure can be nested inside another decision structure

EAST TEXAS A&M

Nested Decision Structures and the `if-elif-else` Statement (2 of 2)

```
MIN_SALARY = 30000.0
MIN_YEARS = 2

if salary >= MIN_SALARY:
    if years_on_job >= MIN_YEARS:
        print('You qualify for the loan.')
    else:
        print(f'You must be employed '
              f'for at least {MIN_YEARS} '
              f'years to qualify.')
else:
    print(f'You must earn at least $'
          f'{MIN_SALARY:,.2f} '
          f''per year to qualify.')
```

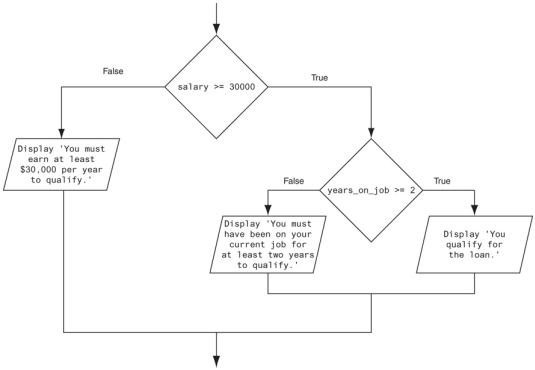


Figure 6: A nested decision structure (Gaddis, 2021, pg.159)

EAST TEXAS A&M

The `if-elif-else` Statement (1 of 2)

- **`if-elif-else` statement**: special version of a decision structure
  - Also called a multi-way decision, or a switch statement
  - Makes logic of nested decision structures simpler to write
  - Can include multiple `elif` statements

```
if condition_1:
    statement(s)
elif condition_2:    # insert as many
    statement(s)     # elif clauses
elif condition_3:    # as necessary
    statement(s)
else:
    statements
```



The if-elif-else Statement (2 of 2)

```
A_SCORE = 90
B_SCORE = 80
C_SCORE = 70
D_SCORE = 60
if score >= A_SCORE:
    print('Your grade is A')
elif score >= B_SCORE:
    print('Your grade is B')
elif score >= C_SCORE:
    print('Your grade is C')
elif score >= D_SCORE:
    print('Your grade is D')
else:
    print('Your grade is F')
```

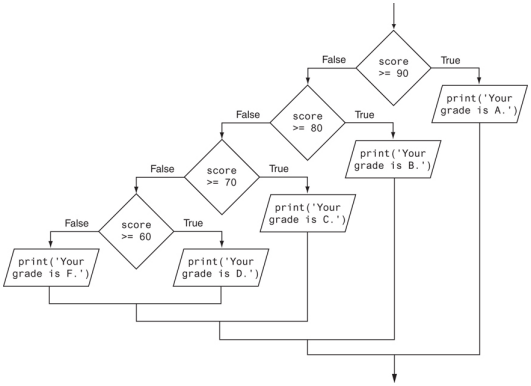


Figure 7: Multi-way decision structure to determine a grade (Gaddis, 2021, pg.162)

EAST TEXAS A&M

Summary

- Use the `if` decision structure to conditionally execute some statement(s) only if a condition is true
- The basic boolean comparison operators like < for less than or >= for greater than or equal to can be used to test numeric conditions
  - The == and != can be used to compare strings
  - < and > and similar will also test string, return alphabetical order
- Use the `if-else` when two possible conditions have actions, some if the condition is `True` but others if the condition is `False`
- You can use, and sometimes need, nested if statements
  - If there are 3 or more conditions, prefer multi-way `if-elif-else` decision structures, reduce level of nesting whenever possible



EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.



EAST TEXAS A&M